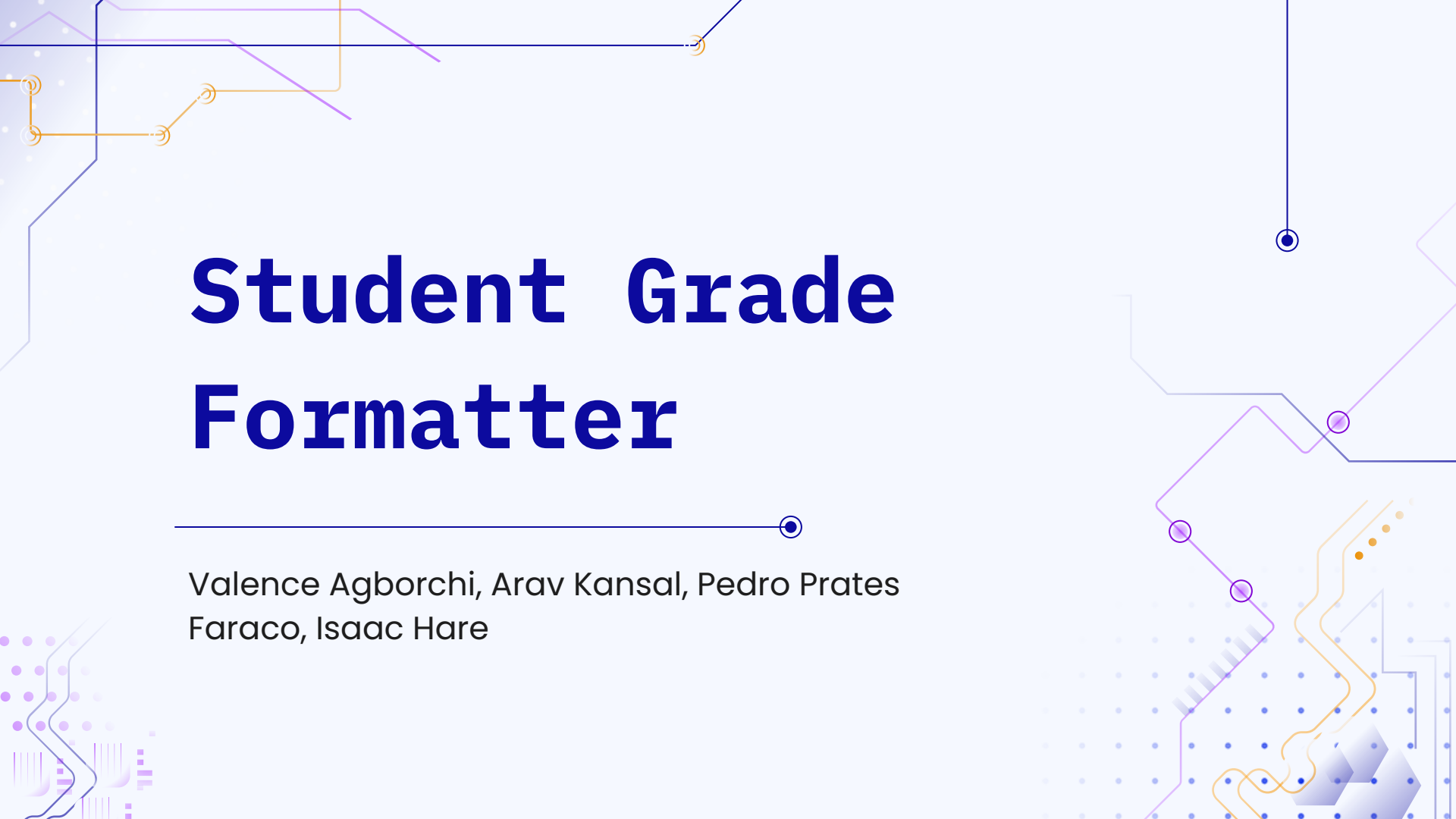


The slide features a light blue background with abstract circuit-like patterns in purple and orange. These patterns include lines, dots, and geometric shapes, primarily located in the top-left, bottom-left, and right-hand corners, framing the central text.

Student Grade Formatter

Valence Agborchi, Arav Kansal, Pedro Prates
Faraco, Isaac Hare

Introduction

Software

- Processes student grade data
- Reads data from two input text files
- Final grades are calculated based on the weighted averages of the data
 - $(\text{test1} + \text{test2} + \text{test3}) * 0.2 + \text{finalExam} * 0.4;$
- A sorted file of formatted results is outputted.

Goals

- Create clear system architecture and detailed design
- Showcase and apply Object Oriented Programming principles
- Offensive Programming (Exception handling, format processing)
- Present system architecture and class relationships in a way that is easy to understand
- Show the reading of input files, grade calculation and proper output through software demonstration

Requirements



Input Text Files

NameFile.txt
CourseFile.txt



Compiling Language

Java



OOP Features

Encapsulation
Polymorphism

Architecture/ Detailed Design

Student class, Course class,
InputHandler class,
GradeCalculator class,
OutputWriter class, and the Main
class



Output (Text File)

OutputFile.txt



Offensive Programming

Format Processing
Exception Handling



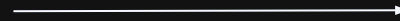
System Architecture Overview



Component-Based System Architecture

- Name.txt and CourseFile.txt flow into InputHandler component
- Data flows through GradeCalculator, OutputWriter then the final produced file, Output.txt
- Main component controls workflow in a Top-Down Approach
 - Delegates task step by step to lower components
 - Acts as a supervisor
- Low Coupling
 - Modules do not rely on each other heavily
- High Cohesion
 - Modules are well related and strictly focused on a single task

User



Main



InputHandler
(Reads Input Files)



NameFile.txt

CourseFile.txt



GradeCalculator
(Computes grades)



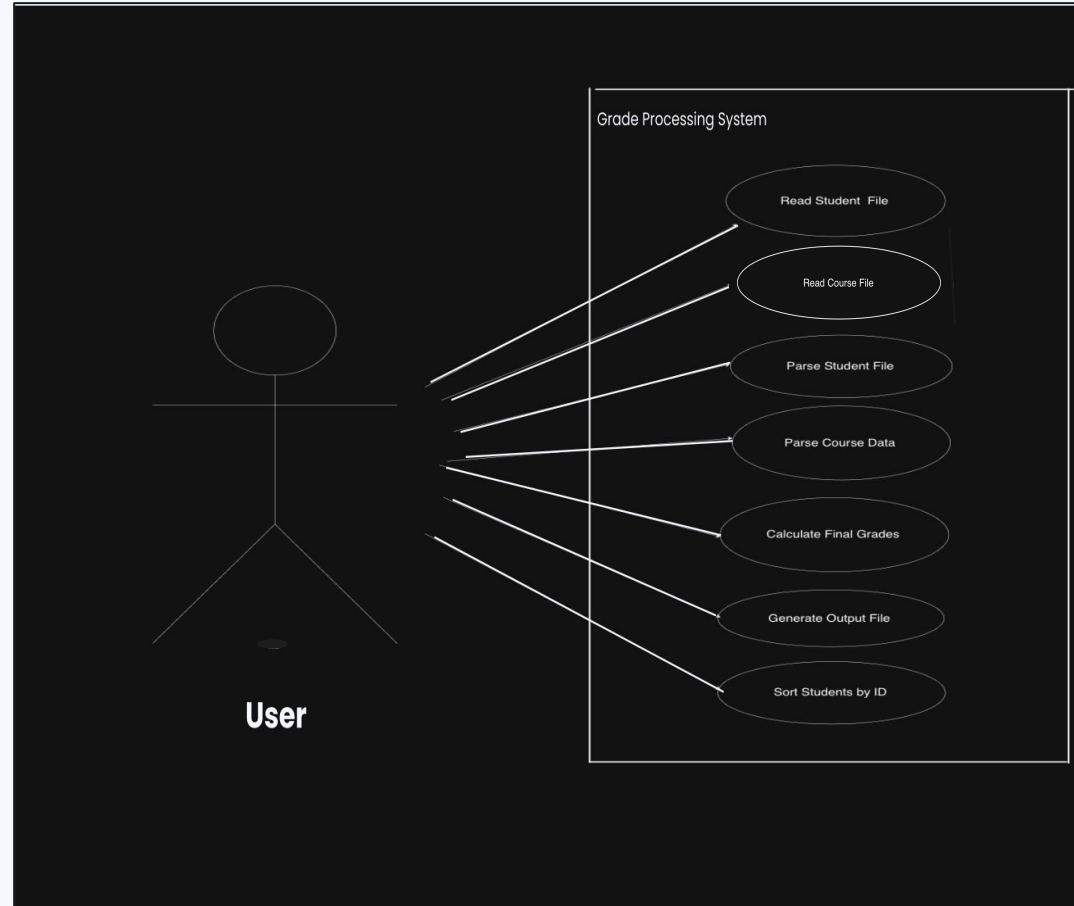
OutputWriter
(Writes File)

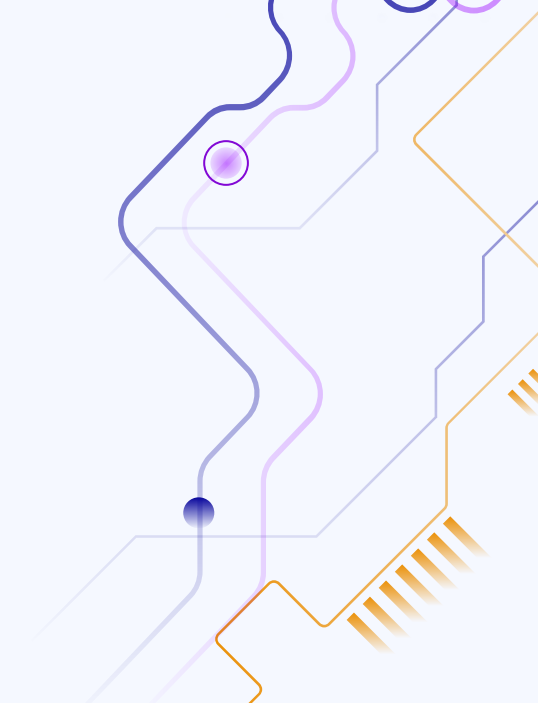
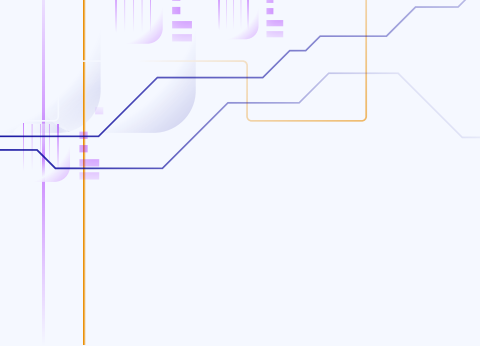


OutputFile.txt

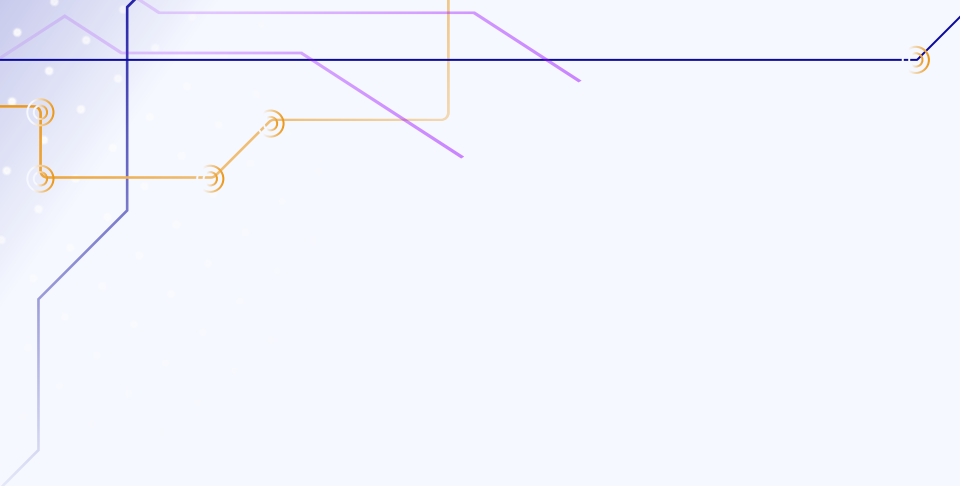
Use Case Diagram

- Interactions between user and the system is shown
- Functional requirements shown at a High level with Actor and Use Cases





Detailed Design Overview

A decorative graphic in the top-left corner featuring a network of thin, dark blue and orange lines. Some lines are straight, while others are angled. Small orange circles are placed at several points where the lines intersect or terminate.

Classes

A decorative graphic in the bottom-left corner consisting of a grid of small, light blue dots. Overlaid on this grid are several thin, dark blue lines that form a series of connected, slightly curved shapes, resembling a stylized circuit or a map.A decorative graphic in the bottom-right corner featuring a grid of small, light blue dots. Overlaid on this grid are several thin, dark blue lines that form a series of connected, slightly curved shapes, resembling a stylized circuit or a map. There are also some orange dots and lines in this area, adding to the circuit-like theme.

Student Class

Stores the name and student ID of each student from the NameFile.txt and the full list of courses each student is enrolled in from CourseFile.txt

Key methods:

- `getStudentID()`
- `getStudentName()`
- `ArrayList<Course>getCourses()`
- `setCourse(Course course)`

Course Class

- Stores course information that is specific to each student which includes student ID, course code, test 1, test 2, test 3 and the final exam

Key methods

calculateFinalGrade(), setCourseID(), getCourseID(), setStudentID(), getStudentID(), setTest1(), getTest1(), setTest2(), getTest2(), setTest3(), getTest3(), setFinalExam(), getFinalExam()

InputHandler Class

- Reads both input files (NameFile.txt and CourseFile.txt) and loads the contents into both the student and course classes

Key methods

- readNameFile()
- readCourseFile()
- setStudentFilePath()
- setCourseFilePath()

GradeCalculator Class

Calculates final marks on test 1, test 2, test 3 and the final exam by calling the `calculateFinalGrade()` method from the `Course` class.

Key method: `computeFinalGrades(ArrayList<Student>)`

OutputWriter Class

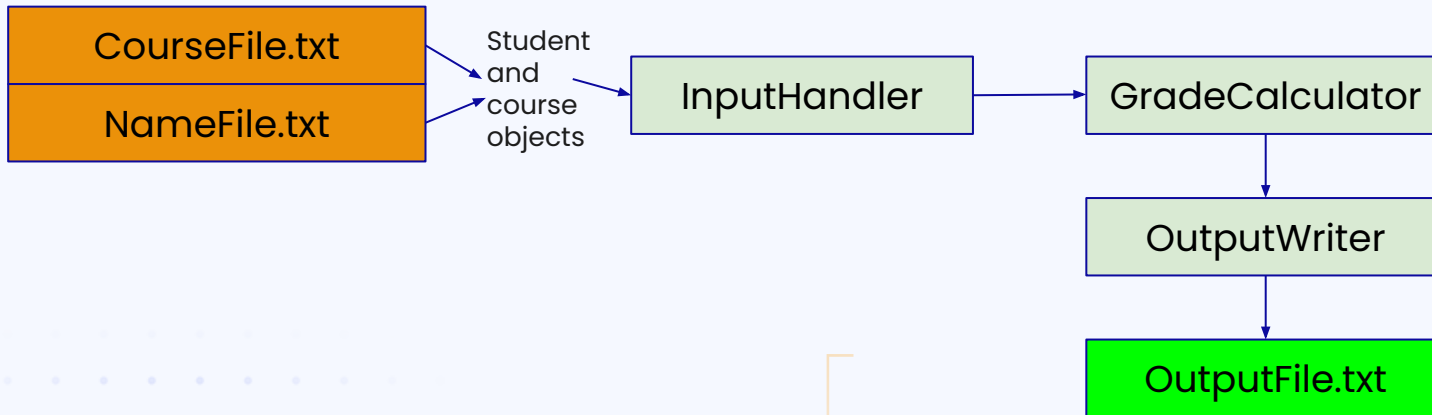
- Writes student ID, name, course code and final mark to a sorted file called OutputFile.txt

Key Methods

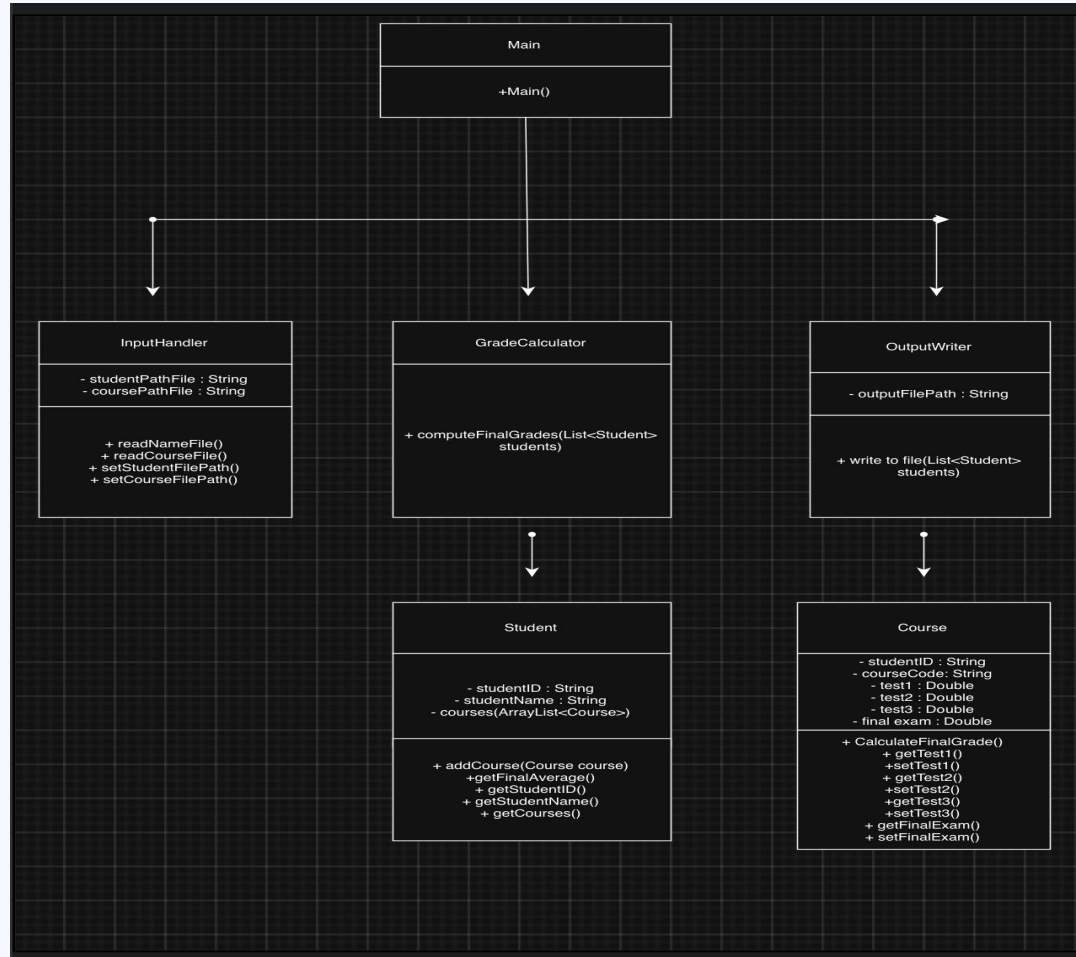
- `setOutputPath(String outputPath)`
- `writeOutputFile(ArrayList<Student> studentList, ArrayList<Course> courseList)`

Main Class

- The class in which the program is run

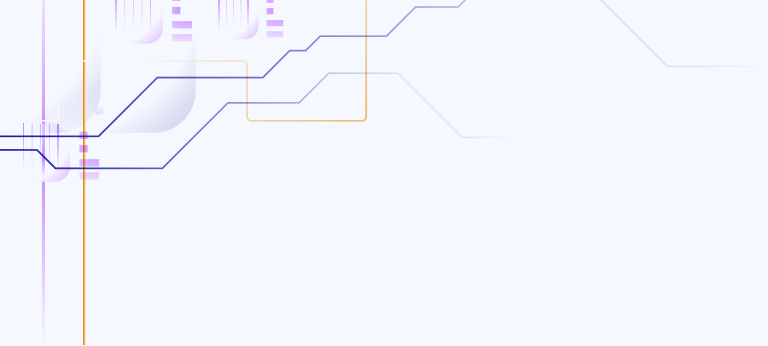


UML Class Diagram



Data Structures: ArrayList

- The ArrayList is used within our Student class, InputHandler class, OutputWriter class and GradeCalculator class
- In the Student class the ArrayList is used to initialize courses as Null so that the list can be filled
- The InputHandler class uses an ArrayList in both the readNameFile() method and the readCourseFile() so that the data from the 2 input files can be dynamically added to a list
- The WriteOutputFile() method in the OutputWriter class contains both an ArrayList for student data and course data as parameters



OOP Features



Encapsulation

OutputWriter Class

```
public class OutputWriter {  
    private String outputPath;
```

InputHandler Class

```
public class InputHandler {  
    private String studentFilePath;  
    private String courseFilePath;
```

Course Class

```
public class Course {  
    private String studentID;  
    private String courseID;  
    private double test1;  
    private double test2;  
    private double test3;  
    private double finalExam;
```

Student Class

```
public class Student {  
    private String studentID;  
    private String studentName;  
    private ArrayList<Course> courses;
```

All variables are made private in respect to their own classes, allowing modification to only take place in their respective classes

Polymorphism

OutputWriter Class

```
OutputWriter() {  
    this.outputPath = "";  
}  
  
OutputWriter(String outputPath) {  
    this.outputPath = outputPath;  
}
```

The InputHandler constructor is overloaded to account for additional string parameters

```
InputHandler() {  
    this.studentFilePath = "";  
    this.courseFilePath = "";  
}  
  
InputHandler(String studentFilePath, String courseFilePath) {  
    this.studentFilePath = studentFilePath;  
    this.courseFilePath = courseFilePath;  
}
```

Examples of compile-time
polymorphism

An Additional Example of Polymorphism

Student Class

```
Student(String studentID, String studentName) {  
    this.studentID = studentID;  
    this.studentName = studentName;  
    this.courses = new ArrayList<>(); //Initializes courses as a null ArrayList that can be filled  
}  
  
Student(String studentID, String studentName, ArrayList<Course> courses) {  
    this.studentID = studentID;  
    this.studentName = studentName;  
    this.courses = courses;  
}
```

Offensive Programming

- The goal of offensive programming is to ensure our software is reliable
- This is done by writing code that doesn't accept bugs or errors
- Our program has a priority of catching bugs early and subsequently throwing an exception
 - NameFile.txt and CourseFile.txt are checked for correctness in formatting before processing
 - Grades are checked to be digits rather than any letters
 - Exception handling is implemented within try/catch blocks in both the InputHandler class and the OutputWriter class

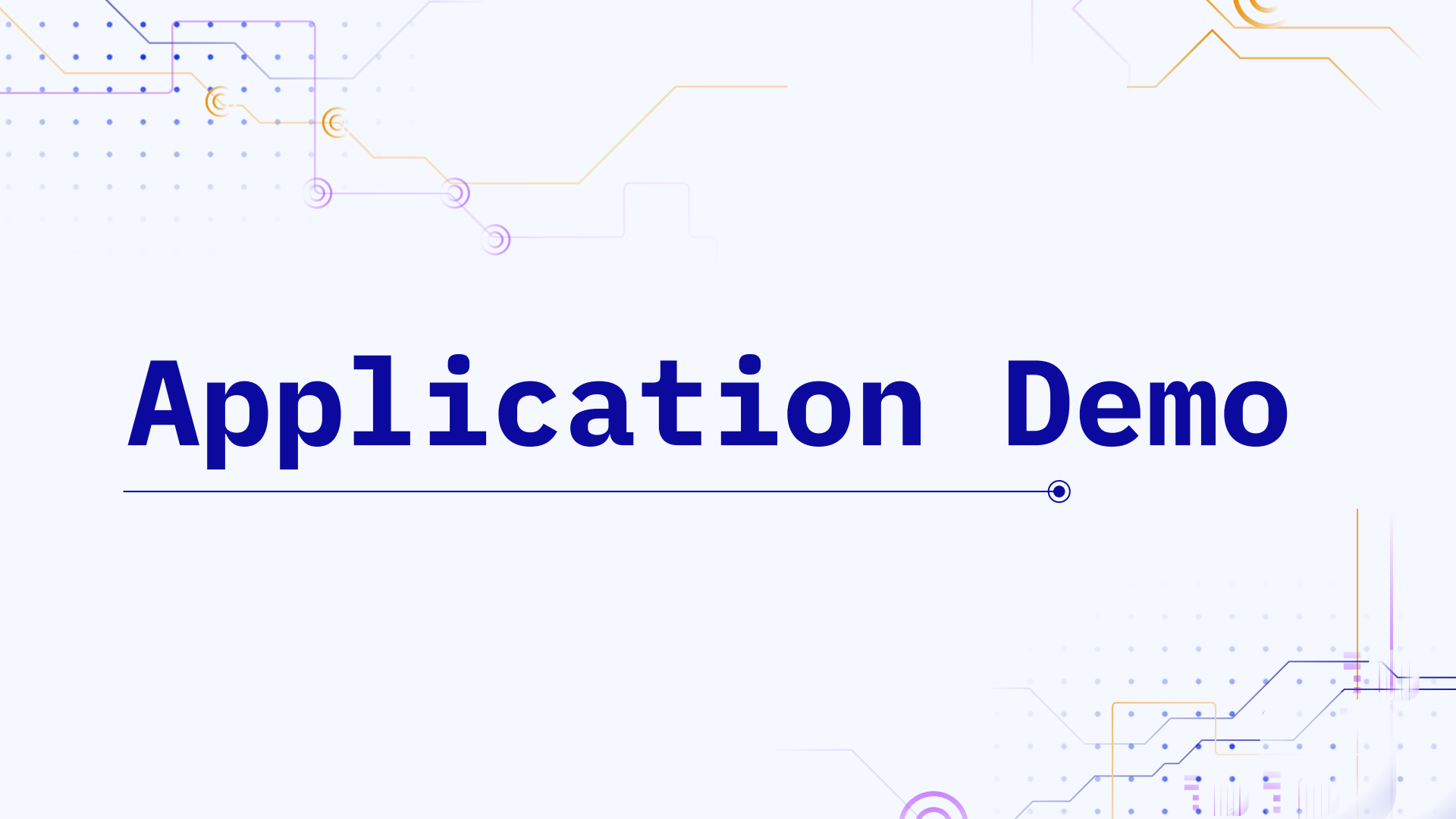
```
InputHandler input = new InputHandler(studentFilePath: "./NameFile.txt", courseFilePath: "./CourseFile.txt");  
try {  
    input.readNameFile(studentList);  
} catch (Exception e) {  
    System.out.println(e.toString());  
}
```

```
void readNameFile(ArrayList<Student> studentList) throws Exception {  
    try {  
        // Read file and load into studentList  
    } catch (Exception e) {  
        // Throw Exception in case of exception  
        throw e;  
    }  
}
```

Offensive Programming

Grades are checked to be digits rather than any letters

```
for (i = 2; i<=5; i++){  
    if (!(courseInfo[i].trim().charAt(0) >='0' && courseInfo[i].trim().charAt(0) <='9')){  
        System.out.println("Error, negative value found");  
        courseList.clear();  
        negative = true;  
    }  
}
```



Application Demo



Conclusion
